The ai-pipeline plugin

Using it, releasing a new version, and installing and updating it in other projects

Source: `plugins/ai-pipeline/` in `pantstamp/ai-dev-pipeline`. Written for Claude Code 2.1.296 on 2026-10-23.

> **Current version: 3.0.1**, tagged `pipeline-v3.0.1`: 3.0.0, the first release as a plugin, with the repository's new name. Each release's changes, and what a project must do to take it, are in `plugins/ai-pipeline/skills/CHANGELOG.md`.

Contents

1. What the plugin is, and where each part lives
2. Using the pipeline in a project
3. Working on the pipeline in this repository
4. Releasing a new version and making it available
5. Installing it in another project
6. Updating and rolling back a project
7. Pitfalls and troubleshooting
8. Quick reference

1. What the plugin is, and where each part lives

A Claude Code **plugin** is a folder of skills and agents packaged as one unit, with a name and a version. The ai-pipeline plugin holds the six skills of the feature pipeline (spec, plan, build, gate, review triage and retrospective) and the two agents they hand work to. Claude Code prefixes everything in a plugin with the plugin's name, so the skills are invoked as `/ai-pipeline:architect` and so on.

```
plugins/ai-pipeline/
|-- .claude-plugin/plugin.json    name "ai-pipeline", version, author
|-- pipeline.template.md          the config a project copies to .claude/pipeline.md
|-- skills/
|   |-- product-owner/  architect/  developer/
|   |-- pre-pr-checklist/  review-triage/  retrospective/
|   |-- PIPELINE_VERSION          the version, stamped on SPEC/PLAN/RETRO
|   |-- CHANGELOG.md              what changed per release, per stage
|   `-- README.md
`-- agents/
    |-- phase-implementer.md
    `-- pre-pr-gate.md

.claude-plugin/marketplace.json   (repo root) lists the plugin and its folder
```

A **marketplace** is the catalogue file `.claude-plugin/marketplace.json`. It tells Claude Code which plugins a git repository offers and where each one lives. Any git repository that has one is a marketplace. Ours offers one plugin, so its full ID is `ai-pipeline@ai-pipeline`: the plugin `ai-pipeline` from the marketplace `ai-pipeline`.

Part	Where it lives	What it is
Source	This repo, `plugins/ai-pipeline/`	Where the pipeline is written and tested, and where releases are tagged
Release	A git tag, `pipeline-vX.Y.Z`	A fixed version that projects can pin
Installed copy	Each developer's machine, `~/.claude/plugins/cache/…`	What Claude Code loads. Read-only, one folder per version
Pin	The consuming project's `.claude/settings.json`, committed	Says which tag the project uses
Project facts	The consuming project's `.claude/pipeline.md`, committed	Branches, commands, hosting: what differs between projects

So a consuming project **does not contain the pipeline**. It holds its own facts (`pipeline.md`), its rules (`CLAUDE.md`) and a pointer to a version. The plugin brings the process and the general Android knowledge, so the same plugin works in any Android project that describes itself in a `pipeline.md`.

2. Using the pipeline in a project

Once the plugin is installed (section 5), each stage is a slash command. Each one ends at a checkpoint where you decide whether to continue. The full walkthrough is `docs/AI_WORKFLOW.md`.

Stage	Command	Produces	You decide
1. Spec	`/ai-pipeline:product-owner`	a branch + `SPEC.md`	Are the acceptance criteria right?
2. Plan	`/ai-pipeline:architect`	`PLAN.md`	The design; you propose it first
3. Build	`/ai-pipeline:developer`	a commit per phase, then the PR	After every phase

Stage	Command	Produces	You decide
4. Gate	`/ai-pipeline:pre-pr-checklist`	GO / CONDITIONAL GO / NO GO	Ship it or fix it
5. Review	`@claude-review` on the PR, then `/ai-pipeline:review-triage`	fixes + thread replies	Which findings to act on
6. Learn	`/ai-pipeline:retrospective`	`RETRO.md` + rule changes	Which lessons to keep

What a project must provide:

- `.claude/pipeline.md`: the project's facts. These are the base branch and prefixes, the feature-docs folder, the host (GitHub or GitLab), the pre-flight command that mirrors CI, the formatter, the architecture-rule tool, lint and the review mechanism. Start from the plugin's `pipeline.template.md` and replace each **Fill in:** value. **Every value is required**, and `none` is allowed only where the table says so. A stage that needs a missing value stops and names it rather than guessing.
- `CLAUDE.md`, or wherever `{{rules}}` points: the project's coding rules. The gate's rule review reads them.
- **The review workflow**, if you want `@claude-review`. It is a CI file (here `.github/workflows/claude-review.yml`), and a plugin cannot ship CI files, so copy it into the project.

> **Which version produced a document?** SPEC, PLAN and RETRO files carry `pipeline: vX.Y.Z` under their title. Retrospectives use it to compare review findings across releases.

3. Working on the pipeline in this repository

This repo plays two roles. It is the **producer**: it holds the source and the test suite, and cuts releases. It is also a **consumer**: it installs the released tag like any other project, so everyday sessions use a stable version. To change the pipeline, load the working copy instead:

```
claude --plugin-dir ./plugins/ai-pipeline
```

- `--plugin-dir` loads the folder as it is on disk, over any installed copy. Run `/reload-plugins` after editing a skill or agent. It works only from the CLI, not from a desktop-app session.
- Commits that touch the pipeline use the prefix `chore(pipeline): ….`
- **Every change adds a line under** `## Unreleased` **in** `plugins/ai-pipeline/skills/CHANGELOG.md`, under its stage, saying what changed and why.
- Skills hold **process**. Project rules go in `CLAUDE.md` and mechanical checks in Konsist or lint. A sentence in a skill must still be true if the feature that prompted it had never existed.
- A skill opens its own files from its base directory, and the project's files only through `.claude/pipeline.md`. Files outside `plugins/ai-pipeline/` are not installed, so a skill must never point outside the plugin folder.

Checks while developing:

```
python3 docs/ai-pipeline/eval.py check     # free: 6 static checks on the pipeline files
claude plugin validate plugins/ai-pipeline # free: the manifest and the folder layout
python3 docs/ai-pipeline/eval.py list      # the golden scenarios
```

`eval.py check` fails, among other things, when a skill names a project-relative `.claude/` path, uses a `{{name}}` that `pipeline.md` lacks, runs a host command outside its **On GitHub** / **On GitLab** section, or when `plugin.json` and `PIPELINE_VERSION` disagree.

4. Releasing a new version and making it available

Choosing the number (semver):

- **Major**: a contract between stages, or a name a user types or a skill spawns, changes. Examples: the PLAN format the gate reads, or the skill names (3.0.0).
- **Minor**: new behaviour in a stage.
- **Patch**: a wording fix.

Steps:

1. **Run the free checks**: `eval.py check` and `claude plugin validate plugins/ai-pipeline`. Both must pass.
2. **Run the golden-scenario suite on the new version, and on the previous release the same day.** Run the previous release from a checkout of its tag; up to 2.1.0 it uses `.claude/`, not the plugin. **The release passes when no scenario is worse than the previous release, run the same day.** Model behaviour drifts, so an old baseline is not proof. A full suite costs about $90 at API prices.
3. **Record the numbers** at the top of each scenario's `RESULTS.md`, with the commit they were taken at.
4. **Open the release PR.** In it, set the version in **both** `plugins/ai-pipeline/skills/PIPELINE_VERSION` and `plugins/ai-pipeline/.claude-plugin/plugin.json`; rename `## Unreleased` in the CHANGELOG to `## X.Y.Z — YYYY-MM-DD`, with an **Upgrading a project** note if projects must change anything; and move this repo's own pin in `.claude/settings.json` to the new tag.
5. **Merge the PR**, then tag the merge commit and push the tag:

```
git switch master && git pull
git tag pipeline-vX.Y.Z           # on the merge commit
git push origin pipeline-vX.Y.Z
```

Pushing the tag is what makes the version available. There is no separate publish step. Because the repository is public, any project can now pin `pipeline-vX.Y.Z` without a token. Then install the new tag in this repo with the update commands in section 6.

> Releases keep the `pipeline-vX.Y.Z` tag scheme rather than `claude plugin tag`'s `ai-pipeline--vX.Y.Z`, because every earlier release and pin already uses it. Never move or re-create a published tag: projects pinned to it would silently get different files.

5. Installing it in another project

Before you start: Claude Code 2.1.280 or newer (`claude --version`), and the target tag must exist on GitHub.

1. **Write the project's `.claude/pipeline.md`.** Copy `plugins/ai-pipeline/pipeline.template.md` from the tag you install (on GitHub, or from the installed copy in `~/.claude/plugins/cache/`) and replace every **Fill in:** value with the project's own. Commit it. It then runs under the project's own CI checks, so it must pass any checks the project applies to its AI-instruction files, such as checks for paths that do not exist.
2. **Remove any copied pipeline**: `.claude/skills/` and `.claude/agents/` in the project, or machine-wide copies in `~/.claude/skills/` and `~/.claude/agents/`. Otherwise the old bare commands (`/architect`) load beside the plugin's.
3. **Each developer runs, once, in the project folder:**

```
claude plugin marketplace add \
  "https://github.com/pantstamp/ai-dev-pipeline.git#pipeline-v3.0.1" \
  --scope project --sparse .claude-plugin plugins

claude plugin install ai-pipeline@ai-pipeline --scope project
```

The first command clones this repository at the tag. With `--sparse` it checks out `.claude-plugin/`, `plugins/` and the repository's top-level files, under 1 MB, so the Android template app's modules are never downloaded. The second copies the plugin into the machine's plugin cache. With `--scope project`, both record themselves in the project's `.claude/settings.json`:

```
{
  "extraKnownMarketplaces": {
    "ai-pipeline": {
      "source": {
        "source": "git",
        "url": "https://github.com/pantstamp/ai-dev-pipeline.git",
        "ref": "pipeline-v3.0.1",
        "sparsePaths": [".claude-plugin", "plugins"]
      }
    }
  },
  "enabledPlugins": { "ai-pipeline@ai-pipeline": true }
}
```

4. **The first developer commits `.claude/settings.json`.** It is the project's record of which version it uses. Teammates still run the two commands above, because the file records the pin but does not install anything.
5. **Restart Claude Code and check:**

```
claude plugin list        # ai-pipeline@ai-pipeline, the expected version, enabled
```

In a session, typing `/ai-pipeline:` should list the six skills. If the project uses `@claude-review`, copy the review workflow file into its CI as well.

6. Updating and rolling back a project

A project updates only when it chooses, never automatically. A pinned tag is not refetched, even with auto-update on.

1. Read the new version's CHANGELOG entry, especially its **Upgrading a project** note. A major version usually needs a change in the project, such as new `pipeline.md` names or new command names. Compare the project's `pipeline.md` with the new version's `pipeline.template.md`: every name in the template must be in the project's file.
2. Open a PR in the project that moves `ref` in `.claude/settings.json` to the new tag and makes any change the note asks for.
3. After it merges, **each developer runs**, with the new tag:

```
claude plugin marketplace add \
  "https://github.com/pantstamp/ai-dev-pipeline.git#pipeline-vX.Y.Z" \
  --scope project --sparse .claude-plugin plugins

claude plugin update ai-pipeline@ai-pipeline --scope project
```

4. Restart Claude Code, then check the version with `claude plugin list`.

Rolling back is the same: a PR moves `ref` back to the old tag, and each developer runs the same two commands with the old tag. The update and the rollback were both tested in both directions.

> **Editing `ref` in `settings.json` does nothing by itself.** Claude Code keeps the pin in a per-user registry (`~/.claude/plugins/known_marketplaces.json`), which reads the settings only when the marketplace is first added. A changed `ref` is ignored by `marketplace update`, `plugin update` and session start. Always re-run `marketplace add` with the tag, then `plugin update`.

7. Pitfalls and troubleshooting

Symptom	Cause and fix
`plugin update` says “already at the latest version” after the `ref` changed	The registry still holds the old tag. Run `marketplace add "<url>#<new tag>"`, then `plugin update`.
A project moved to another version without anyone changing its pin	**One machine has one pin per marketplace name.** Adding `#v2` in project B re-pins the marketplace for every project on that machine, and a later `plugin update` in project A follows it. Installed copies never move by themselves. Before updating, always run `marketplace add` with this project's own tag, never a bare `plugin update`.
`Plugin … not found in marketplace` or `Marketplace … not found`	Settings alone register nothing, and a marketplace declared in settings stays inert in a folder that has never passed the workspace trust dialog. Run the `marketplace add` command.
`/architect` (unprefixed) still appears	A copied pipeline is left in `.claude/` or `~/.claude/`. Delete it (section 5, step 2).
A stage stops and names a missing value	`.claude/pipeline.md` lacks that row, often after a version added a name. Copy the row from the version's `pipeline.template.md` and fill it in.
Edits to the plugin have no effect	Installed copies are cached per version. Use `claude --plugin-dir ./plugins/ai-pipeline` to develop, and `/reload-plugins` after each edit.
`claude -p` fails with 401 or “does not support this model”	`claude auth login`, or `claude update`.

8. Quick reference

Develop (this repo)

```
claude --plugin-dir ./plugins/ai-pipeline    # working copy; /reload-plugins after edits
python3 docs/ai-pipeline/eval.py check       # free static checks
claude plugin validate plugins/ai-pipeline   # manifest and layout
```

Release (this repo)

```
# 1. suite: no scenario worse than the previous release, same day
# 2. release PR: PIPELINE_VERSION + plugin.json + CHANGELOG + own pin
# 3. after merge, on the merge commit:
git tag pipeline-vX.Y.Z && git push origin pipeline-vX.Y.Z
```

Install / update / roll back (any project)

```
claude plugin marketplace add \
  "https://github.com/pantstamp/ai-dev-pipeline.git#<tag>" \
  --scope project --sparse .claude-plugin plugins
# first time:
claude plugin install ai-pipeline@ai-pipeline --scope project
# later versions, forward or back:
claude plugin update ai-pipeline@ai-pipeline --scope project
claude plugin list    # check the version
```

Further reading in the repository: `docs/AI_WORKFLOW.md` (using each stage), `plugins/ai-pipeline/skills/README.md` (skills, agents and versions), `docs/AI_PIPELINE_ROADMAP.md` §2–3 (packaging, versioning, and what was verified), `docs/ai-pipeline/README.md` (the golden-scenario suite).